SENATI

Escuela de Tecnologías de la Información

GUÍA DE LABORATORIO

Solucionario Guía 09

Solucionario: Regresión Lineal

Respuestas paso a paso de los ejercicios propuestos de la Guía 09

Escuela	Tecnologías de la Información
Carrera	Ingeniería de Software con Inteligencia Artificial
Curso	Seminario de Complementación Práctica I (PIAD-427)
Semana	9
Documento	Solucionario para el instructor
Instructor	Rembrandt Ubalde Enríquez

Contenido

Antes de empezar 3
Ejercicio 1. Cambiar test_size a 0.3 4
Paso 1. Entender qué controla test_size 4
Paso 2. Comparar ambos valores en un solo script 4
Paso 3. Analizar los resultados 4
Paso 4. Respuesta explicada 5
Ejercicio 2. Temperatura y ventas de helados 6
Paso 1. Plantear el problema 6
Paso 2. Crear los datos 6
Paso 3. Programa completo (4 etapas) 7
Paso 4. Interpretar los resultados 8
Ejercicio 3. Agregar la variable distrito_premium 9
Paso 1. ¿Qué es una variable binaria (dummy)? 9
Paso 2. Modificar la generación de datos 9
Paso 3. Comparar el modelo con y sin la variable 9
Paso 4. Respuesta explicada 10
Ejercicio 4. Pedir las horas con input() y validar 12
Paso 1. Identificar los casos que hay que controlar 12
Paso 2. Programa 12
Paso 3. Explicación de cada parte 12
Paso 4. Pruebas 13
Ejercicio 5. Comparar con un modelo que predice el promedio 14
Paso 1. ¿Qué es un modelo de referencia (baseline)? 14
Paso 2. Programa 14
Paso 3. Cálculo de la mejora 14
Paso 4. Respuesta explicada 15
Respuestas a las preguntas de reflexión 16

(Si el índice aparece vacío, haz clic derecho sobre él y elige «Actualizar campos».)

Antes de empezar

Todos los ejercicios se resuelven en la misma carpeta de la Guía 09 (**IA_Guia09_Regresion_Lineal**), con el entorno virtual activo. Los ejercicios 1, 4 y 5 necesitan los archivos que crean las prácticas de la guía:

Archivo	Lo crea	Lo usan
`estudiantes.csv`	Práctica 02	Ejercicios 1 y 5
`modelo_notas.pkl`	Práctica 06	Ejercicio 4

Todos los resultados que aparecen en este solucionario se obtuvieron ejecutando el código. Si un estudiante usa otra versión de scikit-learn, los decimales pueden variar ligeramente.

Ejercicio 1. Cambiar test_size a 0.3

Enunciado: Cambia `test_size` a 0.3 en la Práctica 04. ¿Cómo cambian la pendiente y el R^2? Explica por qué.

Paso 1. Entender qué controla test_size

`test_size` indica qué proporción de los datos se separa para la **prueba**. Con 40 estudiantes:

- `test_size=0.2` → 20 % para prueba = **8** estudiantes; 32 para entrenar.
- `test_size=0.3` → 30 % para prueba = **12** estudiantes; 28 para entrenar.

Paso 2. Comparar ambos valores en un solo script

En lugar de editar la Práctica 04 dos veces, un bucle `for` entrena el modelo con cada valor y muestra los resultados en una tabla.

Archivo: ej01_test_size.py

```
# ej01_test_size.py - Comparar test_size = 0.2 vs. 0.3
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

df = pd.read_csv("estudiantes.csv")
X, y = df[["horas_estudio"]], df["nota"]

print("test_size | entren. | prueba | pendiente | intercepto |   R²")
for ts in [0.2, 0.3]:
    X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=ts, random_state=42)
    modelo = LinearRegression().fit(X_tr, y_tr)
    r2 = r2_score(y_te, modelo.predict(X_te))
    print(f"   {ts:<6} | {len(X_tr):^7} | {len(X_te):^6} | {modelo.coef_[0]:^9.4f} |
{modelo.intercept_:^10.4f} | {r2:.3f}")
```

▶ Resultado esperado en la terminal

```
test_size | entren. | prueba | pendiente | intercepto |   R²
   0.2    |   32    |   8    |  1.4619   |   4.5265   | 0.832
   0.3    |   28    |   12   |  1.4447   |   4.5968   | 0.897
```

Paso 3. Analizar los resultados

Indicador	test_size = 0.2	test_size = 0.3	¿Qué pasó?
Pendiente	1.4619	1.4447	Bajó un poco: el modelo aprendió con 4 estudiantes menos.
Intercepto	4.5265	4.5968	Cambió levemente por la misma razón.
R^2	0.832	0.897	Subió, porque se evaluó con **otro grupo** de estudiantes.

Paso 4. Respuesta explicada

- **La pendiente cambia** porque el modelo se entrena con **datos distintos** (28 en vez de 32). Cada grupo de entrenamiento produce una recta ligeramente diferente. El cambio es pequeño (de 1.46 a 1.44) porque la relación entre horas y nota es muy fuerte (correlación 0.963).
- **El R^2 cambia** porque se mide con **otro conjunto de prueba** (12 estudiantes en vez de 8). Estos 12 estudiantes quedaron más cerca de la recta, por eso el R^2 subió a 0.897.
- **Conclusión importante:** que el R^2 haya subido **no significa** que con más datos de prueba el modelo sea mejor. Con datasets pequeños, el R^2 depende mucho de **qué** estudiantes caen en la prueba. Por eso en proyectos reales se usa la **validación cruzada** (`cross_val_score`), que prueba el modelo con varias divisiones y promedia los resultados.

> 💡 **Consejo:** Regla práctica: se suele usar entre 20 % y 30 % para prueba. Menos datos de prueba hace la evaluación poco confiable; menos datos de entrenamiento hace que el modelo aprenda peor.

Ejercicio 2. Temperatura y ventas de helados

Enunciado: Crea un dataset de 30 registros con temperatura (°C) y ventas de helados. Entrena una regresión lineal simple y predice las ventas para 28 °C.

Paso 1. Plantear el problema

- **Variable de entrada (X):** temperatura en °C.
- **Variable objetivo (y):** número de helados vendidos en el día.
- **Tipo de problema:** regresión, porque se predice un **número**.
- **Hipótesis:** a más calor, más ventas (relación lineal positiva).

Paso 2. Crear los datos

Se simulan 30 días con temperaturas entre 18 y 34 °C. Las ventas siguen la regla «12 helados más por cada grado», con un ruido aleatorio que imita la vida real (no todos los días con la misma temperatura se vende igual). `np.random.seed(10)` hace que todos obtengan los mismos datos.

Paso 3. Programa completo (4 etapas)

Archivo: ej02_helados.py

```
# ej02_helados.py - Temperatura vs. ventas de helados
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score

# 1. Preparación de datos: 30 días de verano
np.random.seed(10)
temperatura = np.round(np.random.uniform(18, 34, 30), 1)              # °C
ventas = np.round(12 * temperatura - 150 + np.random.normal(0, 15, 30)) # helados/día
df = pd.DataFrame({"temperatura": temperatura, "ventas": ventas})
df.to_csv("helados.csv", index=False)
print(df.head())
print("Correlación:", round(df["temperatura"].corr(df["ventas"]), 3))

X, y = df[["temperatura"]], df["ventas"]
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42)

# 2. Entrenamiento
modelo = LinearRegression().fit(X_tr, y_tr)
print(f"\nEcuación: ventas = {modelo.intercept_:.2f} + {modelo.coef_[0]:.2f} *
temperatura")

# 3. Predicción para 28 °C
pred28 = modelo.predict(pd.DataFrame({"temperatura": [28]}))[0]
print(f"Ventas estimadas a 28 °C: {pred28:.0f} helados")

# 4. Evaluación
y_pred = modelo.predict(X_te)
print(f"MAE: {mean_absolute_error(y_te, y_pred):.2f} helados | R²: {r2_score(y_te,
y_pred):.3f}")

plt.scatter(X, y, label="Datos")
xs = pd.DataFrame({"temperatura": np.linspace(18, 34, 50)})
plt.plot(xs, modelo.predict(xs), color="red", label="Regresión")
plt.scatter([28], [pred28], color="green", s=120, marker="*", label="Predicción 28 °C")
plt.xlabel("Temperatura (°C)"); plt.ylabel("Helados vendidos")
plt.title("Temperatura vs. ventas de helados"); plt.legend(); plt.grid(alpha=0.3)
plt.savefig("s9_helados.png", dpi=110, bbox_inches="tight")
plt.show()
```

▶ Resultado esperado en la terminal

```
   temperatura  ventas
0         30.3   215.0
1         18.3    91.0
2         28.1   183.0
3         30.0   219.0
4         26.0   158.0
Correlación: 0.984

Ecuación: ventas = -121.54 + 10.96 * temperatura
Ventas estimadas a 28 °C: 185 helados
MAE: 7.60 helados | R²: 0.952
```

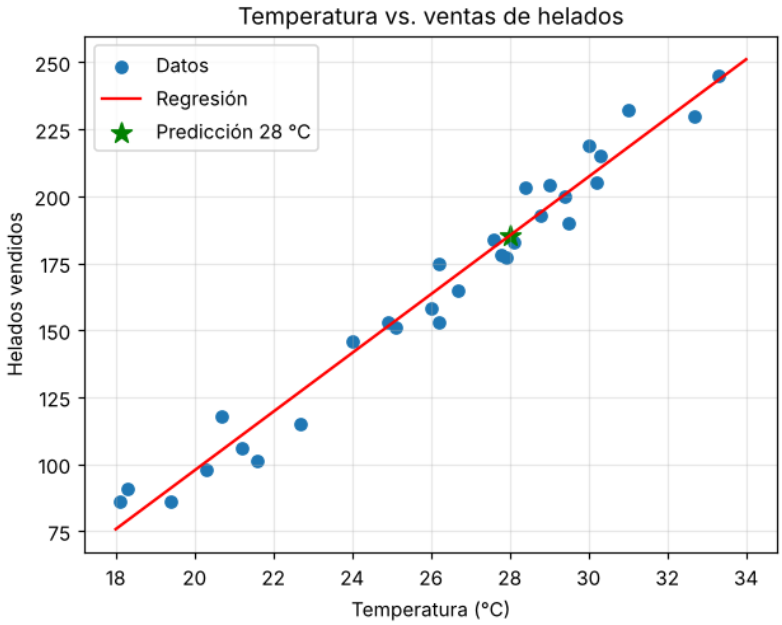


Figura 1. Recta de regresión y predicción para 28 °C (estrella verde).

Paso 4. Interpretar los resultados

Resultado	Valor	Interpretación
Correlación	0.984	Relación lineal positiva muy fuerte: la regresión lineal es adecuada.
Pendiente	10.96	Por cada grado adicional se venden **unos 11 helados más**.
Intercepto	-121.54	No tiene sentido físico (ventas negativas a 0 °C); solo ajusta la recta dentro del rango estudiado (18 a 34 °C).
Predicción a 28 °C	185 helados	-121.54 + 10.96 × 28 ≈ 185.
MAE	7.60	En promedio el modelo se equivoca en unos 8 helados por día.
R^2	0.952	El modelo explica el 95 % de la variación de las ventas.

> ⚠ **Atención:** Un modelo solo es confiable **dentro del rango** de los datos con que se entrenó. Predecir las ventas para 5 °C o 45 °C sería extrapolar y el resultado podría no tener sentido.

Ejercicio 3. Agregar la variable distrito_premium

Enunciado: En la Práctica 05 agrega la variable `distrito_premium` (0 o 1) al cálculo del precio. ¿Mejora el R^2?

Paso 1. ¿Qué es una variable binaria (dummy)?

`distrito_premium` vale **1** si el departamento está en un distrito exclusivo (por ejemplo, Miraflores o San Isidro) y **0** en otro caso. Las variables que solo toman 0 o 1 se llaman **variables dummy** y permiten incluir una característica **categórica** en una regresión lineal.

Paso 2. Modificar la generación de datos

Se agregan dos líneas a la Práctica 05: una que crea la columna y otra que hace que estar en distrito premium **sume S/ 60 000** al precio.

Líneas que se agregan

```
distrito_premium = np.random.randint(0, 2, n)   # 0 o 1
precio = (... + 60000 * distrito_premium + ...)
```

Paso 3. Comparar el modelo con y sin la variable

Se crea la función `evaluar()` que recibe la lista de columnas, entrena y devuelve R^2 y MAE. Así se comparan los dos modelos con **la misma división** de datos.

Archivo: ej03_distrito_premium.py

```
# ej03_distrito_premium.py - ¿Mejora el modelo al agregar distrito_premium?
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error

np.random.seed(7)
n = 120
area = np.random.randint(45, 160, n)
habitaciones = np.random.randint(1, 5, n)
antiguedad = np.random.randint(0, 30, n)
distrito_premium = np.random.randint(0, 2, n)     # 1 = Miraflores, San Isidro...; 0 =
otros
precio = (2500 * area + 15000 * habitaciones - 1800 * antiguedad
          + 60000 * distrito_premium              # estar en distrito premium suma S/
60 000
          + 40000 + np.random.normal(0, 20000, n))
df = pd.DataFrame({"area": area, "habitaciones": habitaciones, "antiguedad":
antiguedad,
                   "distrito_premium": distrito_premium, "precio": precio.round(0)})

def evaluar(columnas):
    X, y = df[columnas], df["precio"]
    X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, random_state=1)
    m = LinearRegression().fit(X_tr, y_tr)
    p = m.predict(X_te)
    return m, r2_score(y_te, p), mean_absolute_error(y_te, p)

base = ["area", "habitaciones", "antiguedad"]
m1, r2_1, mae_1 = evaluar(base)
m2, r2_2, mae_2 = evaluar(base + ["distrito_premium"])
print(f"Sin distrito_premium: R² = {r2_1:.3f} | MAE = S/ {mae_1:,.0f}")
print(f"Con distrito_premium: R² = {r2_2:.3f} | MAE = S/ {mae_2:,.0f}")
print("\nCoeficientes del modelo nuevo:")
for c, v in zip(base + ["distrito_premium"], m2.coef_):
    print(f"  {c:<17}: {v:>10.2f}")
```

▶ Resultado esperado en la terminal

```
Sin distrito_premium: R² = 0.831 | MAE = S/ 34,984
Con distrito_premium: R² = 0.954 | MAE = S/ 17,027

Coeficientes del modelo nuevo:
  area             :    2492.35
  habitaciones     :   13493.39
  antiguedad       :   -1803.17
  distrito_premium :   56755.71
```

Paso 4. Respuesta explicada

- **Sí, mejora mucho:** el R^2 sube de **0.831 a 0.954** y el error promedio (MAE) baja casi a la mitad, de **S/ 34 984 a S/ 17 027**.
- **¿Por qué?** El precio real depende del distrito. Sin esa columna, el modelo no puede explicar por qué dos departamentos iguales tienen precios tan distintos, y ese «misterio» aparece como error.

- **Interpretación del coeficiente:** 56 755.71 significa que, con el mismo área, habitaciones y antigüedad, un departamento en distrito premium cuesta **unos S/ 56 756 más**. Es muy cercano a los S/ 60 000 que se usaron al crear los datos: el modelo «descubrió» la regla.

> ⚠ **Atención:** Agregar una variable solo mejora el modelo si esa variable **realmente influye** en lo que se predice. Si se agrega una columna sin relación (por ejemplo, el color de la puerta), el R^2 casi no cambia.

Ejercicio 4. Pedir las horas con input() y validar

Enunciado: Modifica la Práctica 06 para pedir las horas con `input()` y mostrar un mensaje si el valor no está entre 0 y 12.

Paso 1. Identificar los casos que hay que controlar

Caso	Ejemplo	Qué debe hacer el programa
Número válido	7.5	Predecir la nota y decir si aprueba.
Número fuera de rango	15 o -3	Mostrar «Valor fuera de rango».
Texto que no es número	abc	Mostrar un error, sin que el programa se caiga.

Paso 2. Programa

El modelo ya no se entrena: se **carga** el archivo `modelo_notas.pkl` creado en la Práctica 06.

Archivo: ej04_input_validado.py

```
# ej04_input_validado.py - Pedir horas con input() y validar el rango
import joblib
import pandas as pd

modelo = joblib.load("modelo_notas.pkl")

texto = input("Ingrese las horas de estudio (0 a 12): ")
try:
    horas = float(texto)
except ValueError:
    print("Error: debe escribir un número (use punto decimal, ej. 7.5).")
else:
    if horas < 0 or horas > 12:
        print("Valor fuera de rango: las horas deben estar entre 0 y 12.")
    else:
        nota = modelo.predict(pd.DataFrame({"horas_estudio": [horas]}))[0]
        nota = min(max(nota, 0), 20)          # la nota no puede salir de 0 a 20
        print(f"Con {horas} horas, la nota estimada es {nota:.2f}")
        print("Resultado:", "APRUEBA" if nota >= 10.5 else "DESAPRUEBA")
```

Paso 3. Explicación de cada parte

- `input()` siempre devuelve **texto**. Por eso se convierte con `float(texto)`.
- Si el usuario escribe letras, `float()` lanza un **ValueError**. El bloque `try / except` lo captura y muestra un mensaje en lugar de detener el programa.
- El bloque `else` del `try` se ejecuta **solo si la conversión funcionó**. Ahí se valida el rango con `if horas < 0 or horas > 12`.
- `min(max(nota, 0), 20)` asegura que la nota quede entre 0 y 20, porque una recta puede dar valores fuera de la escala vigesimal.
- Se aprueba con **10.5** o más, como en la escala de SENATI.

Paso 4. Pruebas

▶ **Resultado esperado en la terminal**

```
Ingrese las horas de estudio (0 a 12): 7.5
Con 7.5 horas, la nota estimada es 15.45
Resultado: APRUEBA
```

▶ **Resultado esperado en la terminal**

```
Ingrese las horas de estudio (0 a 12): 2
Con 2.0 horas, la nota estimada es 7.64
Resultado: DESAPRUEBA
```

▶ **Resultado esperado en la terminal**

```
Ingrese las horas de estudio (0 a 12): 15
Valor fuera de rango: las horas deben estar entre 0 y 12.
```

▶ **Resultado esperado en la terminal**

```
Ingrese las horas de estudio (0 a 12): abc
Error: debe escribir un número (use punto decimal, ej. 7.5).
```

💡 **Consejo:** Si el estudiante escribe `7,5` con coma, también aparecerá el mensaje de error, porque Python usa **punto** decimal. Una mejora opcional es usar `float(texto.replace(",", "."))`.

Ejercicio 5. Comparar con un modelo que predice el promedio

Enunciado: Compara el MAE de un modelo que siempre predice el promedio de las notas con el MAE de tu regresión. ¿Cuánto mejora?

Paso 1. ¿Qué es un modelo de referencia (baseline)?

Un **modelo ingenuo** o baseline es la forma más simple de predecir sin usar ninguna variable: responder siempre el **promedio**. Sirve como punto de comparación. Si nuestro modelo no supera al baseline, no está aprendiendo nada útil.

scikit-learn trae este modelo listo: `DummyRegressor(strategy="mean")`.

Paso 2. Programa

Ambos modelos se entrenan y evalúan con **la misma división** (`random_state=42`) para que la comparación sea justa.

Archivo: ej05_baseline.py

```
# ej05_baseline.py - Modelo "ingenuo" (promedio) vs. regresión lineal
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.dummy import DummyRegressor
from sklearn.metrics import mean_absolute_error

df = pd.read_csv("estudiantes.csv")
X, y = df[["horas_estudio"]], df["nota"]
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42)

ingenuo = DummyRegressor(strategy="mean").fit(X_tr, y_tr)   # siempre predice el
promedio
regresion = LinearRegression().fit(X_tr, y_tr)

print(f"El modelo ingenuo siempre predice: {y_tr.mean():.2f}")
mae_ing = mean_absolute_error(y_te, ingenuo.predict(X_te))
mae_reg = mean_absolute_error(y_te, regresion.predict(X_te))
print(f"MAE modelo ingenuo  : {mae_ing:.3f}")
print(f"MAE regresión lineal: {mae_reg:.3f}")
print(f"Mejora: el error baja en {(1 - mae_reg / mae_ing):.1%}")
```

▶ Resultado esperado en la terminal

```
El modelo ingenuo siempre predice: 12.43
MAE modelo ingenuo  : 2.980
MAE regresión lineal: 0.896
Mejora: el error baja en 69.9%
```

Paso 3. Cálculo de la mejora

Fórmula

```
Mejora = 1 - (MAE regresión / MAE ingenuo)
Mejora = 1 - (0.896 / 2.980) = 1 - 0.301 = 0.699  →  69.9 %
```

Paso 4. Respuesta explicada

- Si siempre se predice **12.43**, el error promedio es de casi **3 puntos** de nota.
- La regresión lineal se equivoca en promedio **menos de 1 punto** (0.896).
- El error se reduce en **69.9 %**: las horas de estudio sí ayudan mucho a predecir la nota.
- Esta comparación es justamente lo que mide el **R^2**: compara el error del modelo con el error de predecir siempre el promedio.

Respuestas a las preguntas de reflexión

1. ¿Por qué la regresión lineal es un algoritmo supervisado?

Porque aprende de ejemplos que ya traen la **respuesta correcta** (la etiqueta y). En la Guía 09, cada estudiante tiene sus horas de estudio **y su nota real**; el modelo ajusta la recta comparando sus predicciones con esas notas reales.

2. ¿Qué significa en la vida real una pendiente negativa?

Que cuando la variable de entrada **aumenta**, la variable objetivo **disminuye**. Ejemplo de la Práctica 05: la antigüedad tiene coeficiente -1 665, es decir, cada año de antigüedad reduce el precio del departamento en unos S/ 1 665. Otros ejemplos: más horas en redes sociales → menor nota; más kilómetros recorridos → menor precio de un auto usado.

3. ¿Por qué se evalúa con datos de prueba y no con los de entrenamiento?

Porque el modelo ya «vio» los datos de entrenamiento y se ajustó a ellos; evaluarlo con esos mismos datos sería como tomar un examen con las mismas preguntas del solucionario. Los datos de prueba simulan **datos nuevos** y muestran si el modelo **generaliza**, es decir, si funcionará en la vida real. Así se detecta el **sobreajuste** (overfitting).

4. Dos problemas laborales que se resuelven con regresión lineal múltiple

Respuesta abierta. Ejemplos válidos:

- **Tiempo de desarrollo de software:** estimar las horas de un proyecto según número de pantallas, integraciones y tamaño del equipo.
- **Ventas mensuales de una tienda:** predecirlas según inversión en publicidad, mes del año y número de promociones.
- **Consumo eléctrico de un local:** según temperatura, horas de atención y cantidad de equipos.
- **Tiempo de entrega de un delivery:** según distancia, hora del día y nivel de tráfico.

> 💡 **Consejo:** Al calificar la pregunta 4, verifica que el estudiante nombre **una variable numérica a predecir** y **al menos dos variables de entrada**. Si la salida es una categoría (por ejemplo, «aprobado / desaprobado»), el problema es de clasificación, no de regresión.